

LearnSmart: AI-Powered Personalized Learning Platform

1. Problem Statement

Conventional e-learning platforms treat every learner identically, delivering the same content, pace, and difficulty regardless of a student's actual mastery of underlying concepts. Two specific failures result: students waste time re-studying topics they have already mastered, and students get pushed into topics for which they lack prerequisite mastery, leading to frustration and high drop-off rates.

Most "personalization" offered by existing tools is shallow — typically a single aggregate quiz score, rather than a tracked, per-skill estimate of how well a student knows each underlying concept, how that knowledge evolves over time, or which specific misconception is driving repeated mistakes. Students therefore have no reliable, quantified answer to "what exactly do I not know yet, and what should I study next?"

2. Proposed Solution

We propose **LearnSmart**, a learning platform built around a quantified, per-skill mastery model rather than aggregate quiz scores.

Core mechanism

At the center of the platform is a knowledge tracing model that tracks the probability a student has mastered each individual skill — not just each subject — updating after every question attempt. We use **Bayesian Knowledge Tracing (BKT)** as the baseline mastery model for its interpretability and low data requirement, with a **Deep Knowledge Tracing (DKT, LSTM-based)** model as a planned upgrade once enough interaction data is collected, since DKT can capture skill dependencies that BKT's simpler assumptions miss.

Question difficulty and student ability are jointly calibrated using a simplified **Item Response Theory (IRT)** model, which lets the adaptive quiz engine select the next question to be neither too easy nor too hard for the student's current estimated ability, instead of relying on rule-based difficulty jumps.

Addressing cold start and data availability

A brand-new student has no history, so there is nothing for the model to act on initially. We solve this with a short adaptive diagnostic placement quiz (10–15 questions) at sign-up that initializes the BKT mastery estimate for every skill in the syllabus. To validate and pretrain our models before we have our own user base, we use a public knowledge-tracing benchmark dataset (e.g. ASSISTments or EdNet) to fit baseline parameters per skill category, then fine-tune on real interaction data as our own users generate it.

Explainability

Every recommendation is paired with the mastery estimate that triggered it — for example: "Recommended because your estimated mastery of Binary Search is 41%, and these questions target your most common error pattern in this skill." This directly addresses a common criticism of recommendation systems in education: students and parents don't trust an opaque suggestion. Surfacing the reasoning is something most mainstream platforms don't expose to the end user.

Differentiation from existing players

Khan Academy and Duolingo do model mastery internally, but neither exposes the reasoning behind a recommendation to the learner, and both are closed content ecosystems. BYJU'S personalization leans heavily on engagement analytics (video-watching behavior) rather than skill-level mastery modeling. LearnSmart's differentiation is the combination of (a) transparent, per-skill mastery scores the student can actually see, and (b) a content-agnostic architecture that can ingest any institution's existing question bank rather than requiring our own closed content library — making it usable by colleges and coaching institutes that already have material but lack an adaptive layer.

Feature scope (MVP vs. stretch)

To keep the plan credible for a hackathon timeline, we separate what's actually buildable in the build phase from later additions:

MVP — built during the hackathon:

- Diagnostic placement quiz
- BKT-based mastery tracking per skill
- IRT-based adaptive quiz engine
- Student dashboard with per-skill mastery and explainable recommendations

Stretch goals — if time permits, or post-hackathon:

- DKT model upgrade
 - AI chatbot for doubt resolution
 - Gamification (streaks, badges)
 - Multi-signal learning-style detection from engagement data (e.g. video pause/rewatch patterns, time-on-task), not just quiz scores
-

3. Technology Stack

Architecture overview

Frontend (Next.js) → Node.js/Express API → Python FastAPI ML microservice → MongoDB

We use a hybrid backend: Node.js/Express handles auth, content delivery, and core app logic, while a Python FastAPI microservice exclusively serves the knowledge tracing and IRT models. This lets us use Python's ML ecosystem without forcing the whole backend into Python, and keeps the ML layer independently scalable and replaceable.

Frontend: Next.js, React, TailwindCSS, Recharts/D3 (for mastery-over-time visualizations)

Backend (app layer): Node.js, Express, JWT-based auth

ML microservice: Python, FastAPI, Scikit-learn (BKT parameter fitting via EM), PyTorch (DKT, stretch goal), Pandas, NumPy

Database: MongoDB (profiles, content, attempt logs), Redis (optional — caching mastery state for fast dashboard reads)

Data sources: ASSISTments / EdNet public datasets for offline validation and baseline parameter fitting; synthetic interaction data to stress-test the adaptive engine before real users exist; real user logs collected post-launch for fine-tuning

Deployment: Vercel (frontend), Render/Railway (Node backend + FastAPI service), MongoDB Atlas

4. Expected Impact

Educational impact

Students spend less time on already-mastered material and more on genuine weak points, since recommendations are tied to a quantified mastery score rather than a generic syllabus order. This is measurable directly: time-to-mastery per skill and quiz-score improvement on retested skills are both things we can track and A/B test once a real user base exists.

Social impact

Makes adaptive, skill-level personalization available to any institution that already has its own question bank, without that institution needing to build an in-house ML pipeline. Reduces reliance on expensive one-on-one tutoring purely to diagnose specific weak concepts.

Long-term impact

The content-agnostic design means it could plug into a college's existing LMS or a coaching institute's question bank rather than requiring a closed content library. The underlying knowledge-tracing approach generalizes to other domains — competitive exam prep, professional certification, corporate upskilling.

5. Project Roadmap

Assumption: this roadmap assumes a multi-week build phase typical of multi-round hackathons. If the actual build window is a single 24–48 hour hack, the MVP scope in Section 2 is the version meant to fit that timeline.

Phase 1 (Week 1) — Foundation: finalize the skill graph for a pilot subject; scaffold Next.js + Node.js + MongoDB; source and clean a public knowledge-tracing dataset for offline validation.

Phase 2 (Week 1–2) — Core ML: fit BKT parameters on the public dataset; build the diagnostic placement quiz generator; build the IRT-based adaptive question selector.

Phase 3 (Week 2) — Integration: connect the FastAPI ML microservice to the Node backend; build the student dashboard (mastery view + explainable recommendations); implement attempt-logging for future fine-tuning.

Phase 4 (Week 3) — Testing: internal testing with synthetic and small real user groups; validate that mastery estimates track actual quiz performance sensibly; UI/UX pass.

Phase 5 (Week 3–4) — Demo & beyond: deployment and demo prep; stretch features if time allows (DKT upgrade, chatbot, gamification); post-hackathon, pursue a pilot with a college course or coaching batch for real-world data collection.

Conclusion

LearnSmart's bet is narrow and specific: personalization should be driven by a transparent, per-skill mastery model, not a black-box content filter. By naming the actual mechanism (BKT/DKT, IRT-based difficulty calibration), addressing the cold-start problem with a diagnostic quiz and public-dataset pretraining, and staying content-agnostic so it can sit on top of any institution's existing material, the platform aims to be both technically credible and practically adoptable.